

OOT_设计原则

SOLID 总览

缩写	原则	一句话理解
SRP	Single Responsibility Principle	一个类应该只有一个改变的理由
OCP	Open-Closed Principle	对扩展开放，对修改关闭
LSP	Liskov Substitution Principle	子类必须能够替换父类使用
ISP	Interface Segregation Principle	接口要小而专，不要强迫客户端依赖不必要的方法
DIP	Dependency Inversion Principle	依赖抽象，不依赖具体实现

1. SRP 单一职责原则

小题背景

一个图形系统中，`Rectangle` 类既负责计算矩形面积，又负责在 GUI 上绘制矩形。计算几何程序只需要面积计算，而图形界面程序需要绘制功能。现在 GUI 绘制方式变化时，计算几何程序也可能被迫重新编译和测试。

Bad Java Code

```
class Rectangle {
    private double width;
    private double height;

    public double area() {
        return width * height;
    }

    public void draw(GuiCanvas canvas) {
        canvas.drawRectangle(width, height);
    }
}

class GeometryApp {
    public double calculateArea(Rectangle rectangle) {
        return rectangle.area();
    }
}
```

```
    }  
}  
  
class GraphicalApp {  
    public void render(Rectangle rectangle, GuiCanvas canvas) {  
        rectangle.draw(canvas);  
    }  
}
```

违反的原则

违反 **SRP: Single Responsibility Principle**，单一职责原则。

为什么违反

`Rectangle` 有两个职责：

1. 表示矩形的几何模型，并计算面积；
2. 在 GUI 上绘制矩形。

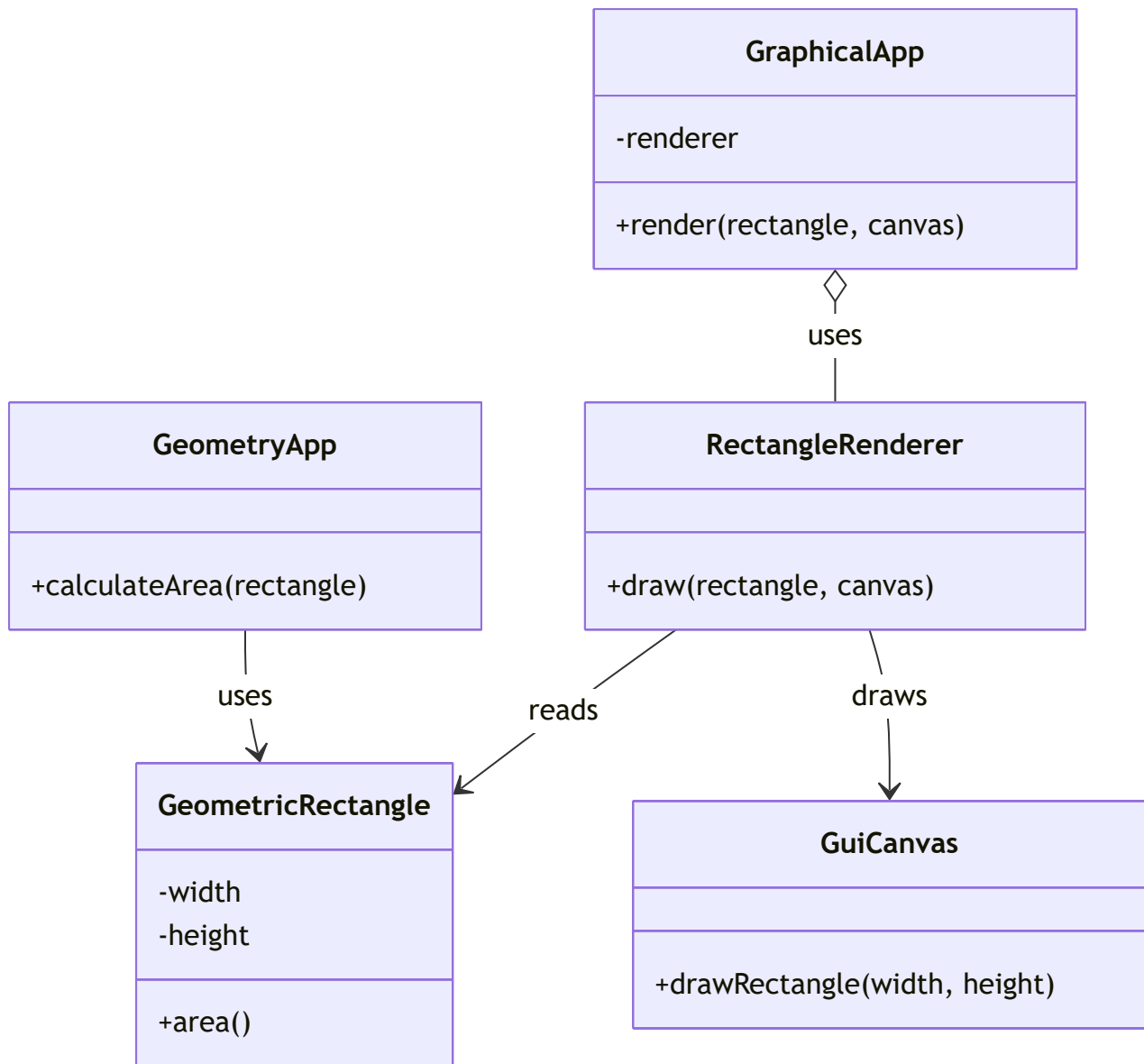
这意味着它有两个改变理由：几何计算规则变化，或者 GUI 绘制方式变化。如果 GUI 绘制变化导致 `Rectangle` 修改，那么只依赖几何计算的 `GeometryApp` 也会受到影响。

重构方案

使用 **Extract Class**。将几何计算职责和图形绘制职责分离：

- `GeometricRectangle`：只负责矩形数据和面积计算；
- `RectangleRenderer`：只负责绘制矩形。

重构后 Mermaid UML



答题模板

该设计违反 SRP。`Rectangle` 同时承担几何计算和 GUI 绘制两个职责，因此有多个改变理由。GUI 绘制方式变化时，几何计算相关代码也可能受到影响。可以将几何模型和绘制逻辑分离，提取 `GeometricRectangle` 负责面积计算，提取 `RectangleRenderer` 负责绘制。

2. OCP 开闭原则

小题背景

一个电商系统根据订单类型计算折扣。当前代码在 `OrderService` 中使用 `if / else` 判断订单类型。现在系统经常新增折扣类型，例如学生折扣、节日折扣、会员折扣。

Bad Java Code

```
class Order {
    private String type;
```

```

private double totalAmount;

public String getType() {
    return type;
}

public double getTotalAmount() {
    return totalAmount;
}
}

class OrderService {
    public double calculatePrice(Order order) {
        if (order.getType().equals("NORMAL")) {
            return order.getTotalAmount();
        } else if (order.getType().equals("VIP")) {
            return order.getTotalAmount() * 0.8;
        } else if (order.getType().equals("FESTIVAL")) {
            return order.getTotalAmount() * 0.7;
        }
        return order.getTotalAmount();
    }
}

```

违反的原则

违反 **OCP: Open-Closed Principle**，开闭原则。

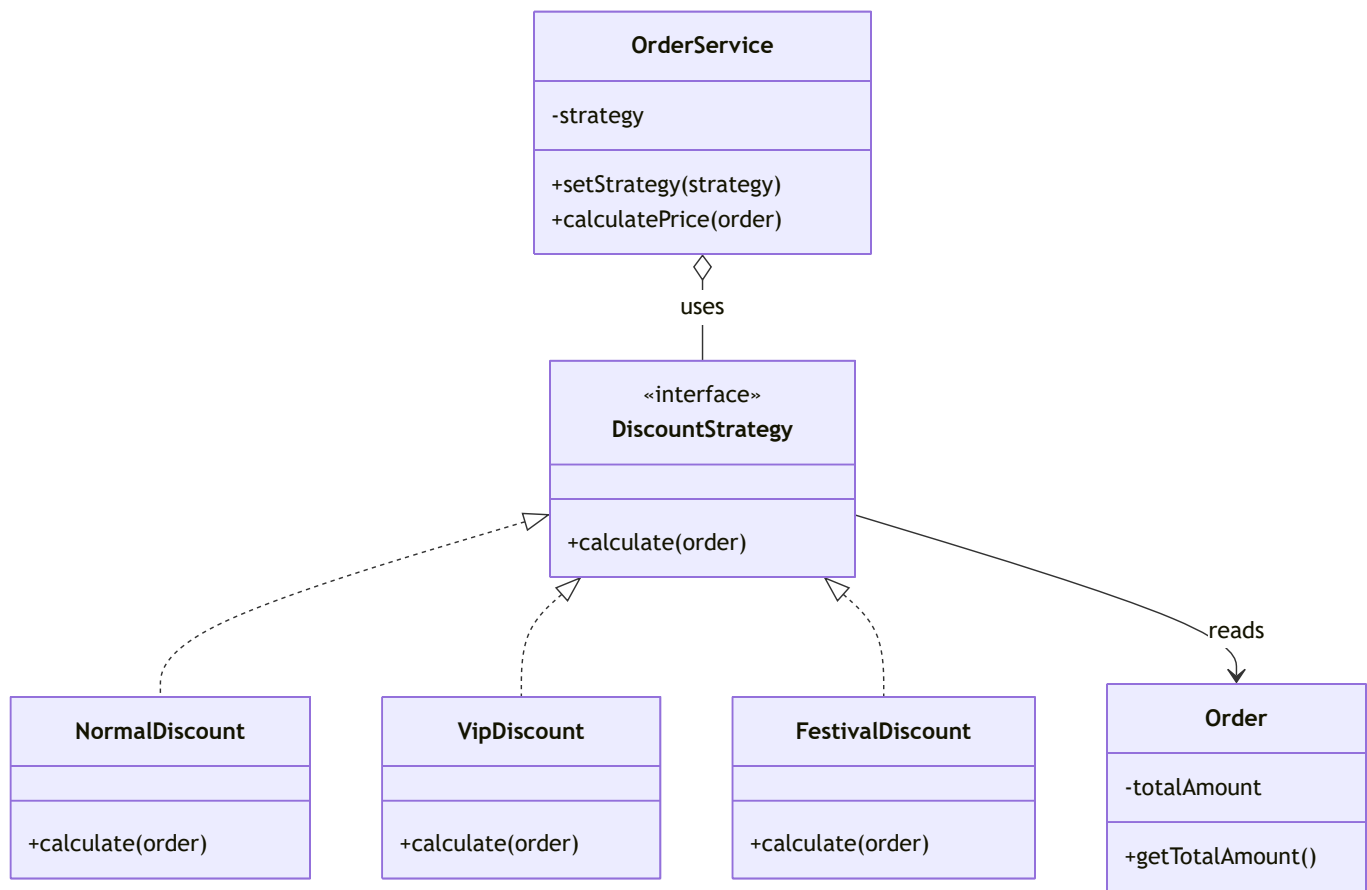
为什么违反

每增加一种折扣类型，都必须修改 `OrderService.calculatePrice()`。系统不是通过扩展新类支持新行为，而是通过修改旧代码支持新行为。这会导致原有代码经常被改动，容易引入 bug。

重构方案

使用 **Strategy Pattern / Replace Conditional with Polymorphism**。将不同折扣规则抽象为 `DiscountStrategy` 接口，每种折扣一个具体策略类。

重构后 Mermaid UML



答题模板

该设计违反 OCP。`OrderService` 通过 `if / else` 判断订单类型，每增加一种折扣类型都要修改原有方法。可以将折扣计算抽象为 `DiscountStrategy` 接口，不同折扣实现不同策略类。重构后新增折扣时只需要增加新的策略类，不需要修改 `OrderService`。

3. LSP 里氏替换原则

小题背景

系统中有 `Rectangle` 和 `Square`。程序员认为“正方形是一种矩形”，于是让 `Square` 继承 `Rectangle`。但是 `Rectangle` 允许分别设置宽和高，而 `Square` 必须保持宽高相等。

Bad Java Code

```
class Rectangle {
    protected double width;
    protected double height;

    public void setWidth(double width) {
        this.width = width;
    }
}
```

```

    public void setHeight(double height) {
        this.height = height;
    }

    public double area() {
        return width * height;
    }
}

class Square extends Rectangle {
    @Override
    public void setWidth(double width) {
        this.width = width;
        this.height = width;
    }

    @Override
    public void setHeight(double height) {
        this.height = height;
        this.width = height;
    }
}

class GeometryTest {
    public void checkArea(Rectangle rectangle) {
        rectangle.setWidth(5);
        rectangle.setHeight(4);

        if (rectangle.area() != 20) {
            throw new RuntimeException("Bad area");
        }
    }
}

```

违反的原则

违反 **LSP: Liskov Substitution Principle**，里氏替换原则。

为什么违反

`checkArea(Rectangle rectangle)` 期望任何 `Rectangle` 都可以独立设置宽和高：

```

setWidth(5)
setHeight(4)

```

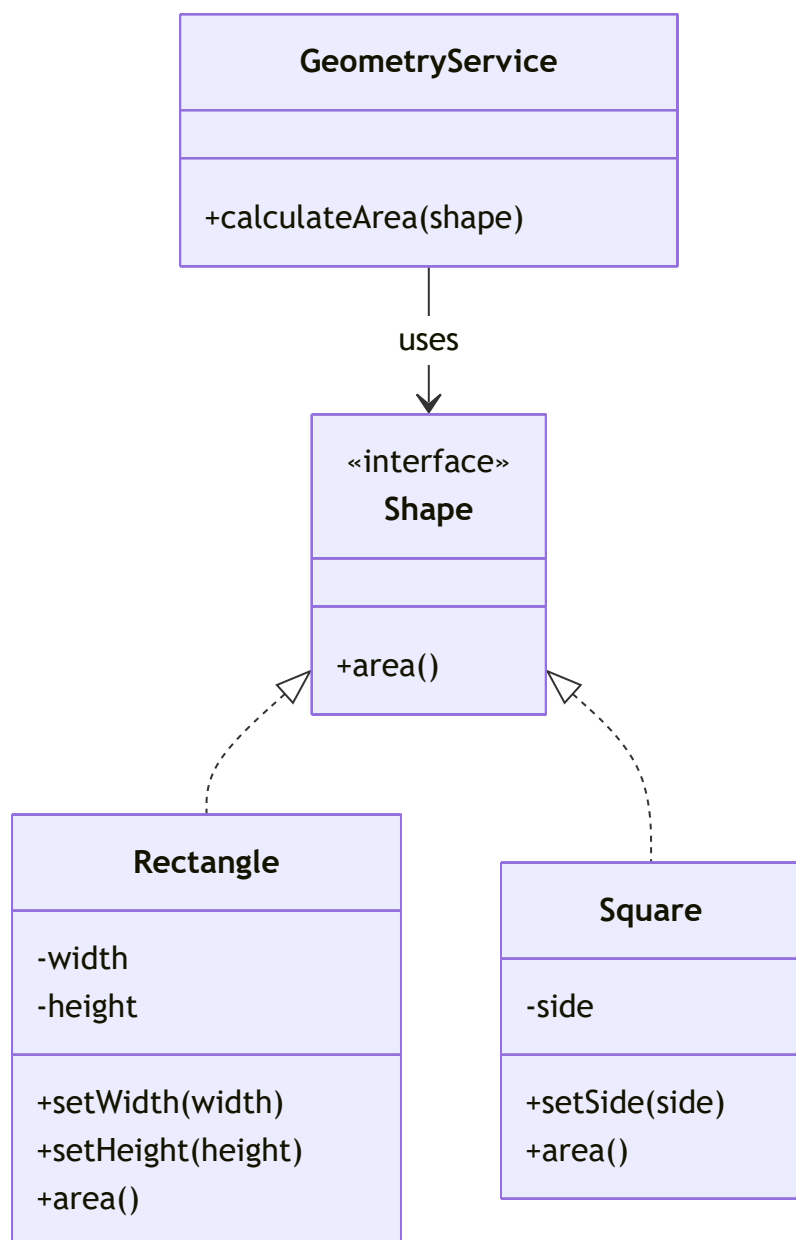
```
area() == 20
```

普通矩形满足这个行为，但如果传入 `Square`，`setHeight(4)` 会同时修改宽和高，最后面积变成 16。所以 `Square` 不能正确替换 `Rectangle`。虽然数学上正方形是矩形，但程序行为上它们不满足同一个父类契约。

重构方案

不要让 `Square` 继承 `Rectangle`。提取共同抽象 `Shape`，让 `Rectangle` 和 `Square` 都实现 `area()`。

重构后 Mermaid UML



答题模板

该设计违反 LSP。`Square` 继承 `Rectangle` 后，不能满足 `Rectangle` 可以独立设置宽和高的行为约定。因此，当需要 `Rectangle` 的地方传入 `Square`，程序行为会出错。应取消这种继承关系，提取共同的 `Shape` 接口，让 `Rectangle` 和 `Square` 分别实现 `area()`。

4. ISP 接口隔离原则

小题背景

系统中定义了一个 `Worker` 接口，要求所有工人都能工作和吃饭。后来系统加入机器人，机器人可以工作，但不需要吃饭。为了实现接口，`RobotWorker.eat()` 只能抛异常。

Bad Java Code

```
interface Worker {
    void work();
    void eat();
}

class HumanWorker implements Worker {
    public void work() {
        // human works
    }

    public void eat() {
        // human eats
    }
}

class RobotWorker implements Worker {
    public void work() {
        // robot works
    }

    public void eat() {
        throw new UnsupportedOperationException("Robot does not eat");
    }
}
```

违反的原则

违反 ISP: Interface Segregation Principle, 接口隔离原则。

为什么违反

`Worker` 接口太大，把 `work()` 和 `eat()` 强行放在一起。`RobotWorker` 只需要 `work()`，却被迫实现 `eat()`。这说明接口没有面向具体客户端拆分。

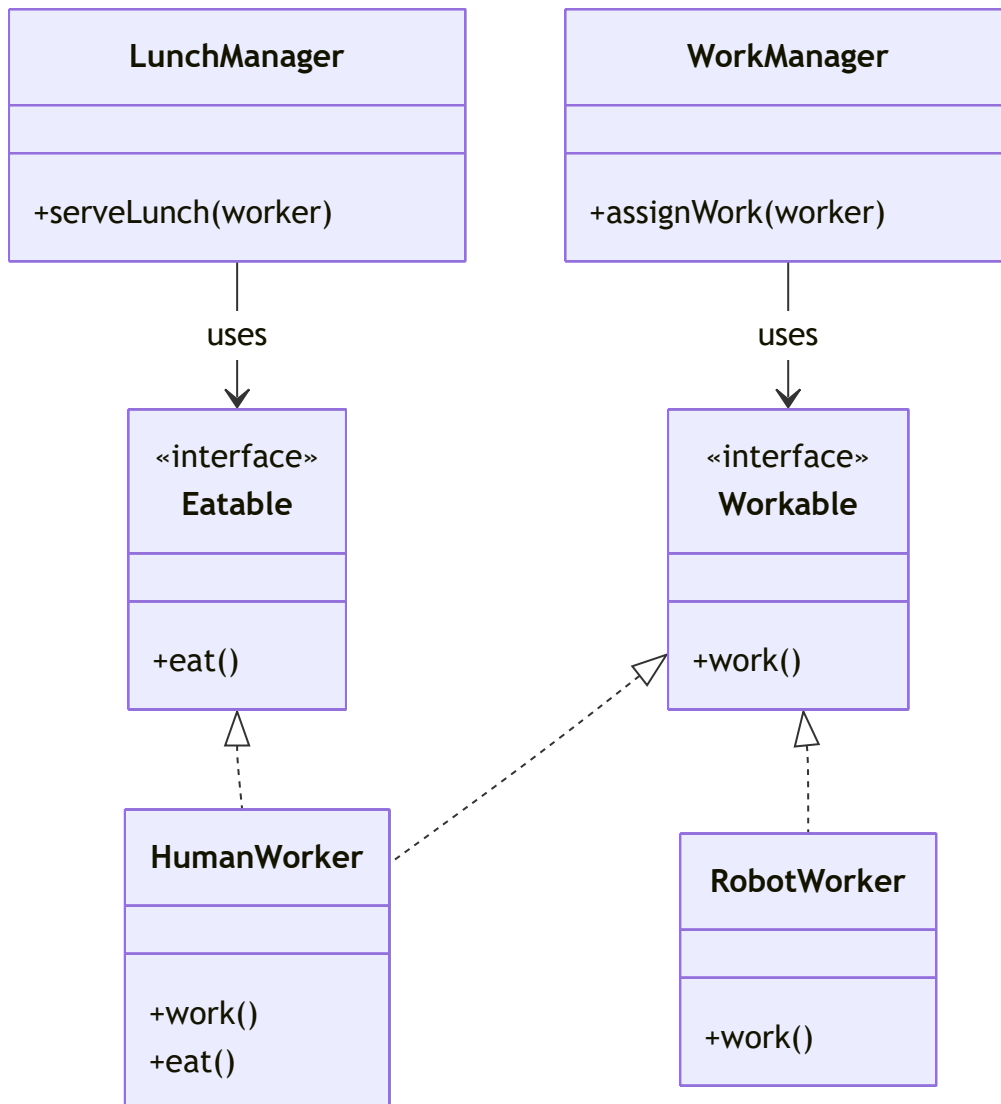
重构方案

拆分接口：

- `Workable`：表示能工作；
- `Eatable`：表示能吃饭。

人类工人同时实现两个接口，机器人工人只实现 `Workable`。

重构后 Mermaid UML



答题模板

该设计违反 ISP。`Worker` 接口包含了某些实现类不需要的方法，导致 `RobotWorker` 被迫实现 `eat()` 并抛出异常。应将大接口拆分为小接口，例如 `Workable` 和 `Eatable`。这样客户端只依赖自己需要的接口，实现类也只实现自己真正支持的行为。

5. DIP 依赖倒置原则

小题背景

订单系统创建订单后需要发送通知。当前 `OrderService` 在内部直接创建 `EmailSender`，所以只能发送邮件。后来系统需要支持短信通知、App 推送，并且测试时希望使用 Mock 对象。

Bad Java Code

```
class EmailSender {  
    public void send(String message) {  
        // send email  
    }  
}  
  
class OrderService {  
    private EmailSender sender = new EmailSender();  
  
    public void createOrder() {  
        // create order  
        sender.send("Order created");  
    }  
}
```

违反的原则

违反 DIP: Dependency Inversion Principle，依赖倒置原则。

为什么违反

`OrderService` 是高层业务模块，`EmailSender` 是低层实现细节。现在高层模块直接依赖具体低层类，并且直接 `new EmailSender()`。

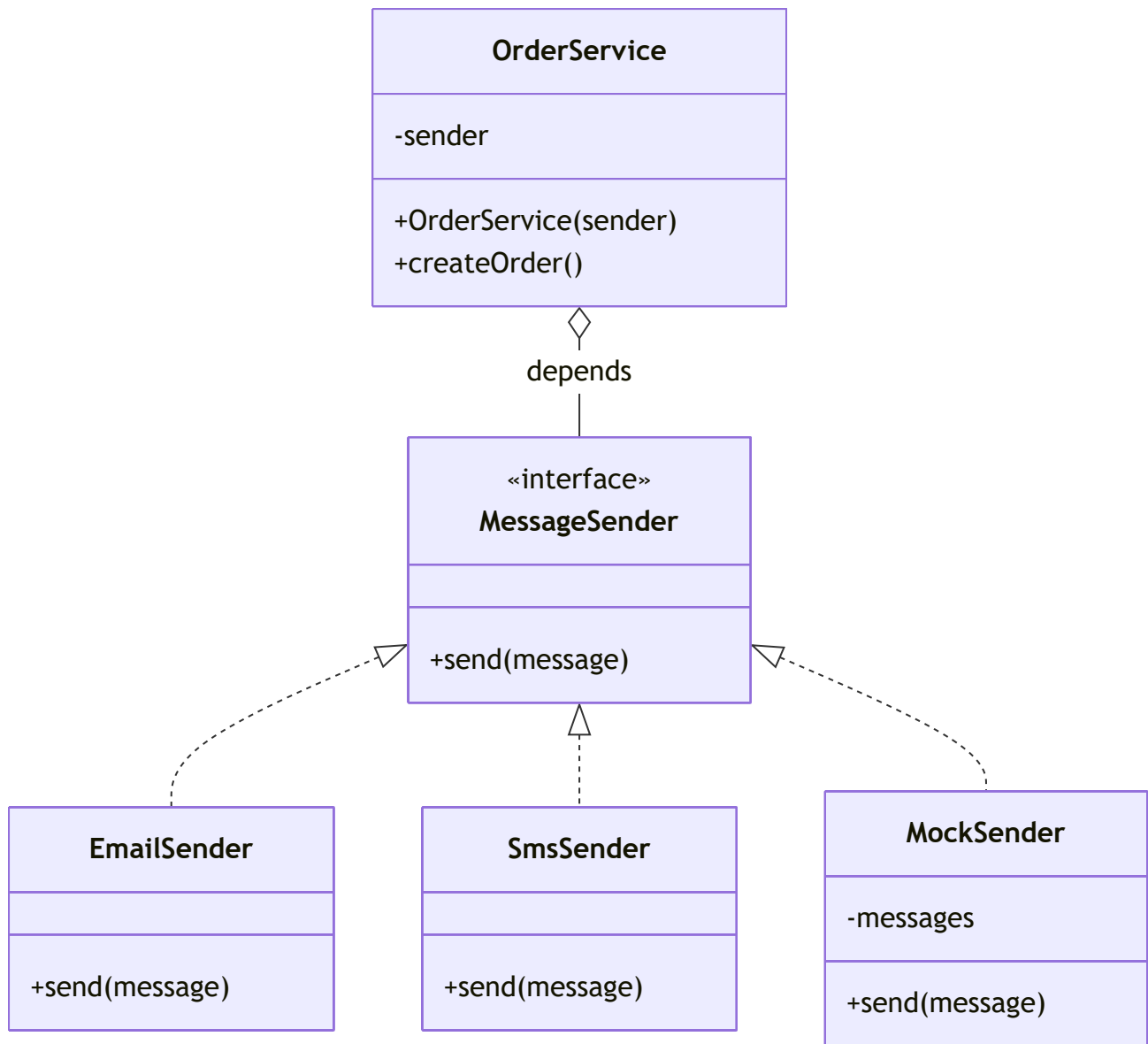
问题：

1. 如果改成短信通知，需要修改 `OrderService`；
2. 测试时不能轻松替换成 `MockSender`；
3. 高层业务逻辑和底层通知实现耦合；
4. 违反“依赖抽象，不依赖具体实现”。

重构方案

定义 `MessageSender` 接口，让 `OrderService` 依赖抽象接口。具体发送方式由外部注入。

重构后 Mermaid UML



Refactored Java Sketch

```
interface MessageSender {
    void send(String message);
}

class EmailSender implements MessageSender {
    public void send(String message) {
        // send email
    }
}

class SmsSender implements MessageSender {
    public void send(String message) {
        // send sms
    }
}
```

```

class OrderService {
    private MessageSender sender;

    public OrderService(MessageSender sender) {
        this.sender = sender;
    }

    public void createOrder() {
        // create order
        sender.send("Order created");
    }
}

```

答题模板

该设计违反 DIP。高层模块 `OrderService` 直接依赖具体低层类 `EmailSender`，导致通知方式难以替换，也不利于测试。应定义 `MessageSender` 抽象接口，让 `OrderService` 依赖抽象，并通过构造函数注入具体实现。这样运行时可以注入 `EmailSender` 或 `SmsSender`，测试时可以注入 `MockSender`。

6. 综合例题：一个代码可能同时违反多个原则

小题背景

支付系统中，`PaymentService` 负责根据支付类型执行支付。它直接在方法里创建不同支付对象，并用 `if / else` 判断支付类型。

Bad Java Code

```

class AliPay {
    public void pay(double amount) {
        // pay by AliPay
    }
}

class WeChatPay {
    public void pay(double amount) {
        // pay by WeChat
    }
}

class PaymentService {
    public void pay(String type, double amount) {
        if (type.equals("ALI")) {

```

```
        AliPay aliPay = new AliPay();
        aliPay.pay(amount);
    } else if (type.equals("WECHAT")) {
        WeChatPay weChatPay = new WeChatPay();
        weChatPay.pay(amount);
    }
}
```

违反的原则

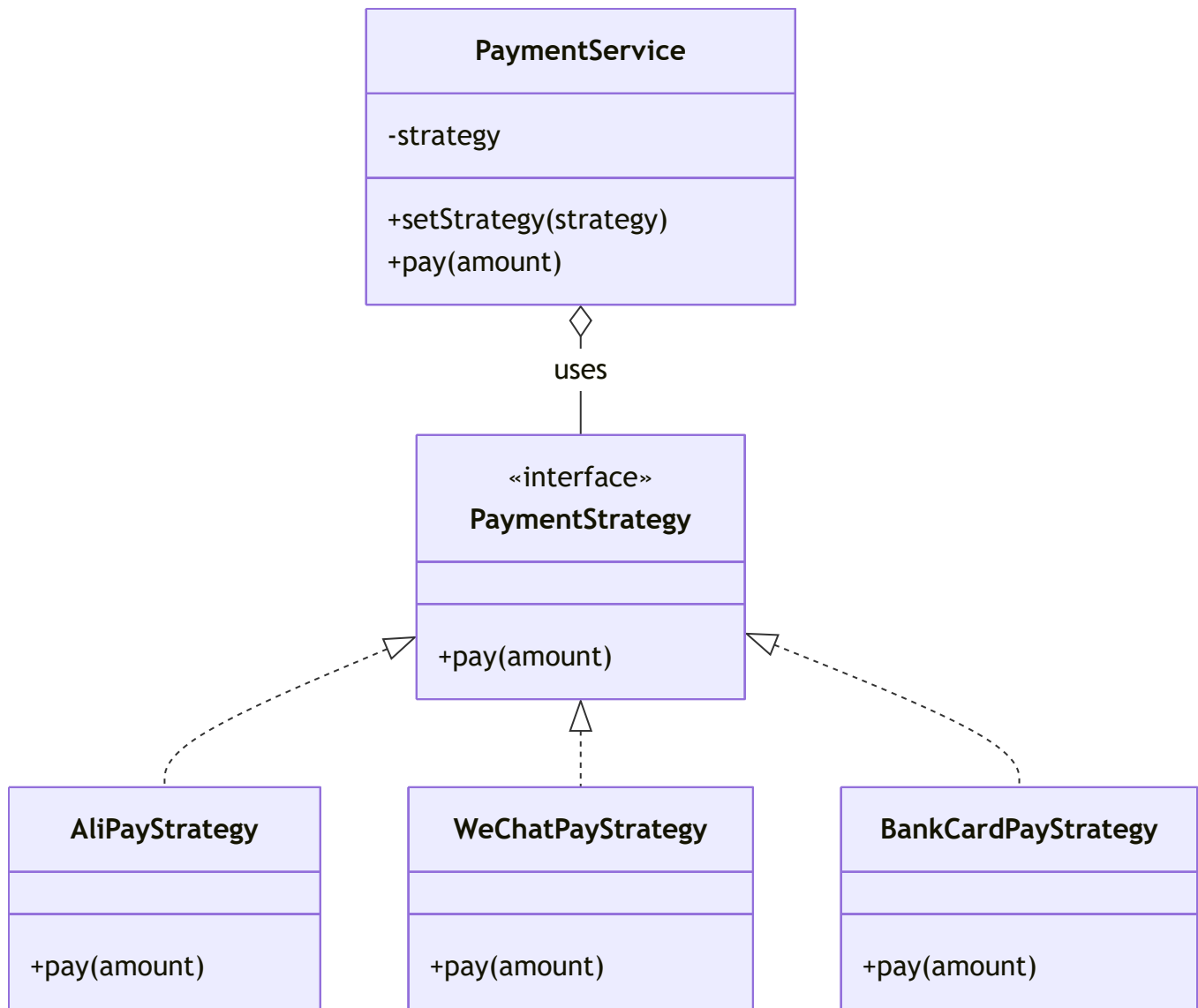
这段代码至少违反：

1. **OCP**：新增支付方式时需要修改 `PaymentService`；
2. **DIP**： `PaymentService` 直接依赖具体支付类 `AliPay`、 `WeChatPay`；
3. 可能违反 **SRP**： `PaymentService` 同时负责选择支付方式和执行支付流程。

重构方案

使用 `PaymentStrategy` 接口，将不同支付方式封装为策略类。 `PaymentService` 只依赖 `PaymentStrategy` 抽象。

重构后 Mermaid UML



答题模板

该代码违反 OCP 和 DIP。`PaymentService` 通过 `if / else` 判断支付类型，每新增一种支付方式都需要修改原有代码，违反开闭原则。同时它直接创建 `AliPay`、`WeChatPay` 等具体类，违反依赖倒置原则。可以抽象出 `PaymentStrategy` 接口，让不同支付方式实现该接口，`PaymentService` 只依赖抽象策略。重构后新增支付方式只需要新增策略类。

7. SOLID 快速识别表

代码 / UML 现象	优先判断
一个类同时负责业务、数据库、UI、日志等多个职责	SRP
一个类有多个改变理由	SRP
新增功能必须修改旧代码	OCP
大量 <code>if / else</code> 或 <code>switch</code> 判断类型	OCP

代码 / UML 现象	优先判断
子类重写方法后破坏父类行为约定	LSP
子类方法抛 <code>UnsupportedOperationException</code>	LSP 或 ISP
代码频繁判断对象真实类型 <code>instanceof</code> / <code>type</code>	LSP，也可能 OCP
一个接口包含太多方法	ISP
实现类被迫实现自己不需要的方法	ISP
高层模块直接 <code>new</code> 低层具体类	DIP
业务类直接依赖数据库、邮件、支付等具体类	DIP
测试时难以替换依赖	DIP

8. SOLID 和 Bad Smell 的对应关系

Bad Smell	常见违反原则
Large Class / Poor Cohesion	SRP
Long Method	SRP
Switch Statements	OCP
Parallel Case Statements	OCP
Refused Bequest	LSP
Inappropriate Inheritance	LSP
Fat Interface	ISP
Direct Concrete Dependency	DIP
Global Variables / Singleton Abuse	DIP / 可测试性问题
Shotgun Surgery	SRP / OCP
Feature Envy	SRP / 高内聚问题

9. 考试简答模板

SRP 模板

该设计违反 SRP。类中包含多个职责，因此有多个改变理由。一个职责变化时会影响另一个无关职责。可以使用 Extract Class，将不同职责拆分到不同类中，使每个类只有一个清晰的改变理由。

OCP 模板

该设计违反 OCP。新增功能时必须修改已有代码，说明系统没有对修改关闭。可以引入抽象接口和多态，通过新增子类或策略类扩展行为，而不是修改原有类。

LSP 模板

该设计违反 LSP。子类虽然继承了父类，但不能在行为上替代父类对象。使用父类的代码如果传入该子类，会出现错误或异常。应重新设计继承结构，提取更合适的抽象，保证子类满足父类的行为契约。

ISP 模板

该设计违反 ISP。接口过大，包含某些客户端不需要的方法，导致实现类被迫依赖或实现无关方法。应将大接口拆分为多个小接口，让客户端只依赖自己真正需要的接口。

DIP 模板

该设计违反 DIP。高层模块直接依赖低层具体实现，导致耦合强、难以替换和测试。应定义抽象接口，让高层模块依赖抽象，并通过依赖注入传入具体实现。